UNIVERSITY OF ENGINEERING AND TECHNOLOGY, PESHAWAR

Department of Computer Science

DATA STRUCTURE LAB

Lab Task 5: Linked Lists

Pointers • Structures • Dynamic Memory • Nodes • Chaining • Traversal

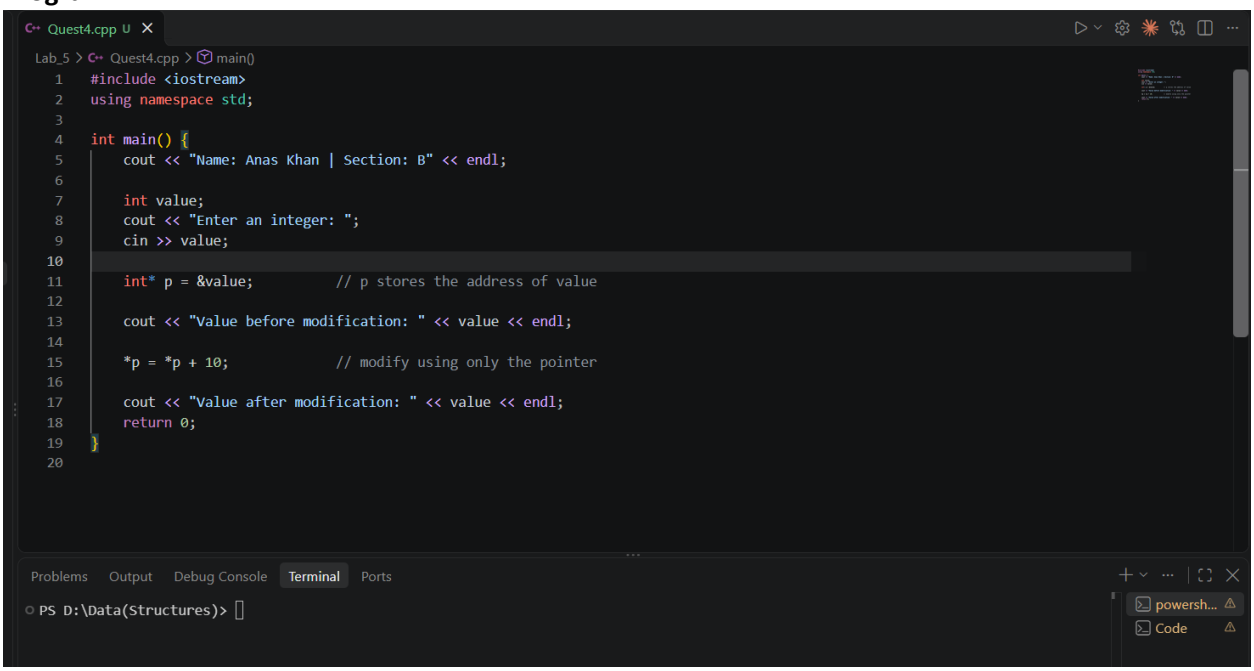
---

Submitted by	Anas Khan
Registration No.	1691
Section	B
Program	Computer Science
Course	Data Structure Lab
Week / Lab	Week 5 / Lab Task 5
Submitted to	Sir Mudassir Ahmad Khan
University	University of Engineering and Technology, Peshawar

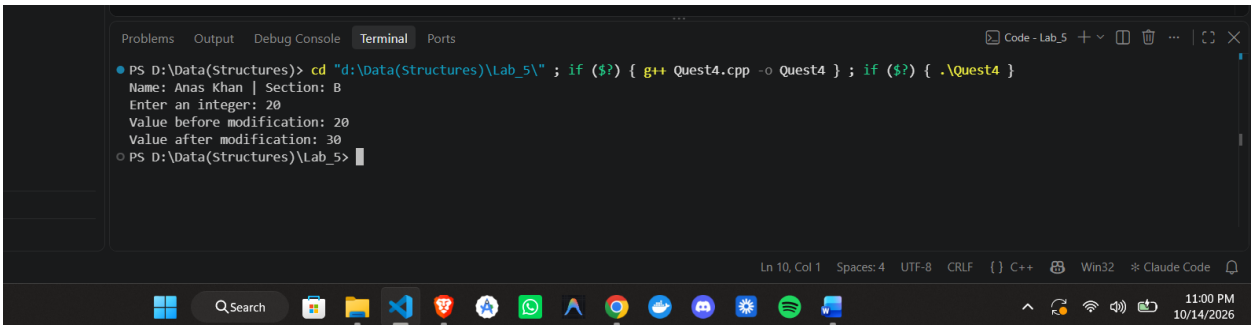
TASK 1: Modify a Value Through a Pointer

Task Statement: Read an integer into a variable, create a pointer to it, then add 10 to the value by using only the pointer for the modification. Display the value before and after modification.

Program:



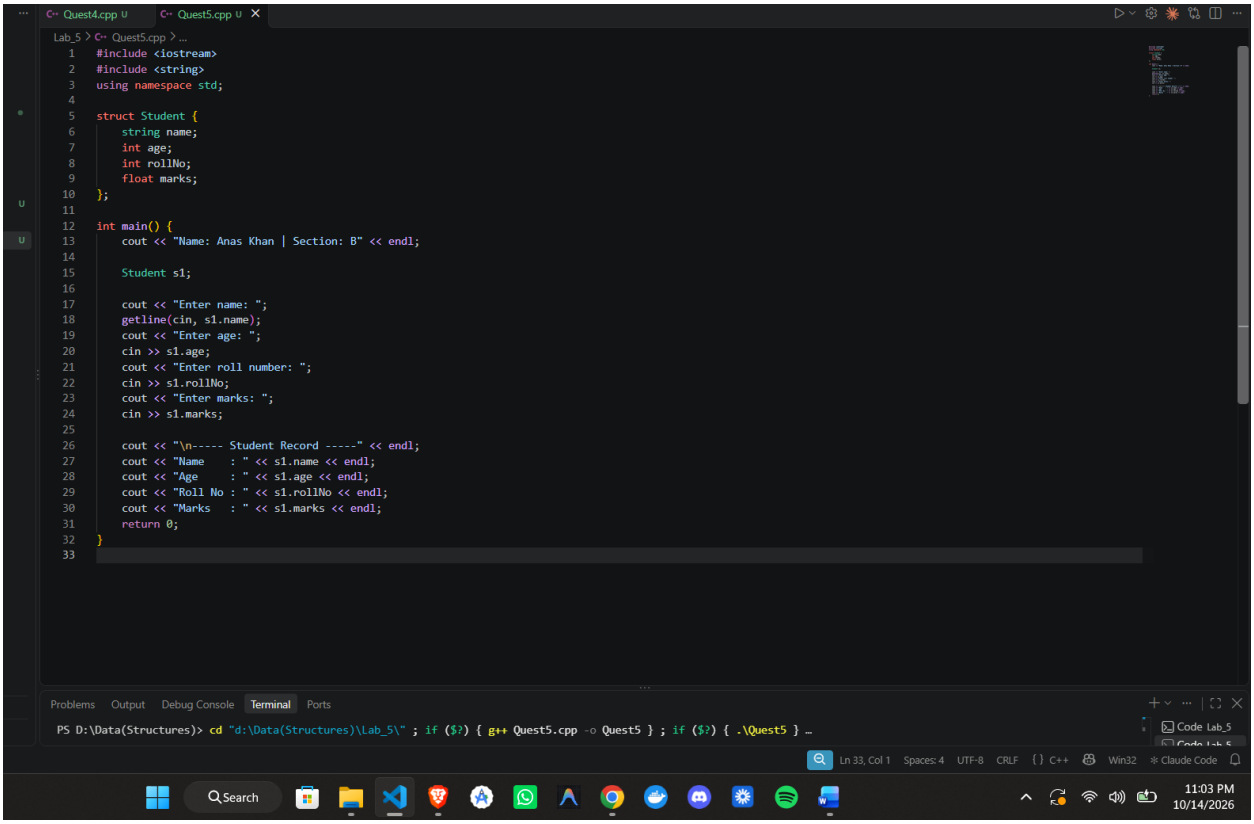
Console Output Screenshot:



TASK 2: Create and Display a Student Structure

Task Statement: Define a Student with name, age, rollNo, and marks. Take the values from the user and display the complete record.

Program:



```cpp
#include <iostream>
#include <string>
using namespace std;

struct Student {
    string name;
    int age;
    int rollNo;
    float marks;
};

int main() {
    cout << "Name: Anas Khan | Section: B" << endl;

    Student s1;

    cout << "Enter name: ";
    getline(cin, s1.name);
    cout << "Enter age: ";
    cin >> s1.age;
    cout << "Enter roll number: ";
    cin >> s1.rollNo;
    cout << "Enter marks: ";
    cin >> s1.marks;

    cout << "\n----- Student Record -----" << endl;
    cout << "Name    : " << s1.name << endl;
    cout << "Age     : " << s1.age << endl;
    cout << "Roll No : " << s1.rollNo << endl;
    cout << "Marks   : " << s1.marks << endl;
    return 0;
}
```

Output Screenshot:



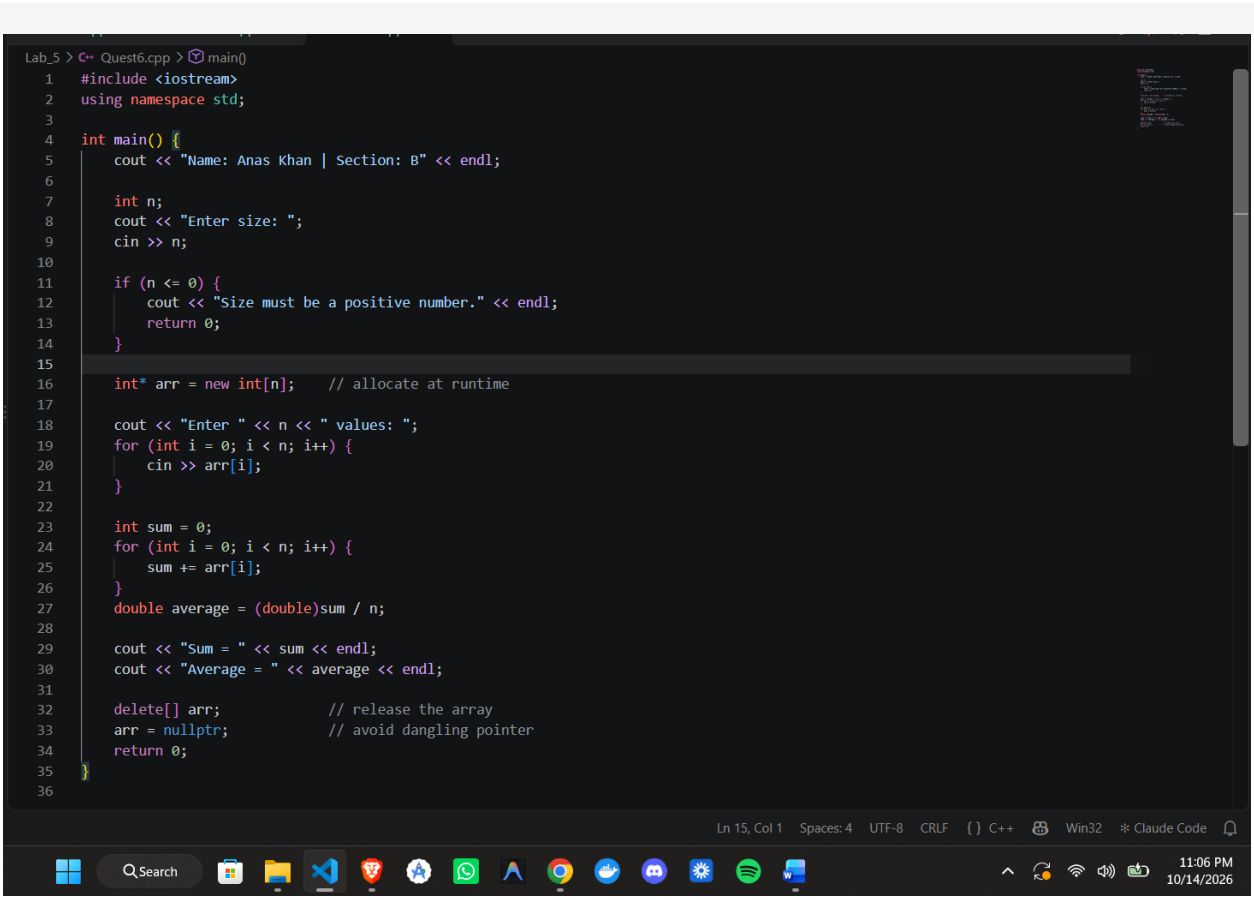
```
PS D:\Data(Structures)\Lab_5> cd "d:\Data(Structures)\Lab_5\" ; if ($?) { g++ Quest5.cpp -o Quest5 } ; if ($?) { .\Quest5 }
Name: Anas Khan | Section: B
Enter name: Azha
Enter age: 23
Enter roll number: 2792
Enter marks: 33

----- Student Record -----
Name    : Azha
Age     : 23
Roll No : 2792
Marks   : 33
PS D:\Data(Structures)\Lab_5>
```

TASK 3: Dynamic Array with Average

Task Statement: Ask the user for n, allocate an integer array dynamically, input n values, calculate their average, then release the memory.

Program:



```cpp
#include <iostream>
using namespace std;

int main() {
    cout << "Name: Anas Khan | Section: B" << endl;

    int n;
    cout << "Enter size: ";
    cin >> n;

    if (n <= 0) {
        cout << "Size must be a positive number." << endl;
        return 0;
    }

    int* arr = new int[n];    // allocate at runtime

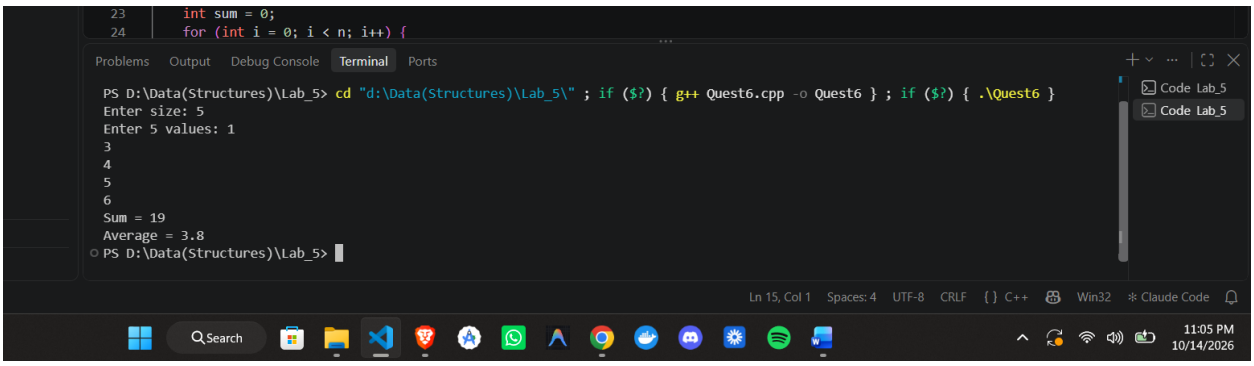
    cout << "Enter " << n << " values: ";
    for (int i = 0; i < n; i++) {
        cin >> arr[i];
    }

    int sum = 0;
    for (int i = 0; i < n; i++) {
        sum += arr[i];
    }
    double average = (double)sum / n;

    cout << "Sum = " << sum << endl;
    cout << "Average = " << average << endl;

    delete[] arr;             // release the array
    arr = nullptr;            // avoid dangling pointer
    return 0;
}
```

Output Screenshot:



```
PS D:\Data(Structures)\Lab_5> cd "d:\Data(Structures)\Lab_5\" ; if ($?) { g++ Quest6.cpp -o Quest6 } ; if ($?) { .\Quest6 }
Enter size: 5
Enter 5 values: 1
3
4
5
6
Sum = 19
Average = 3.8
PS D:\Data(Structures)\Lab_5>
```

TASK 4: Represent an Empty Linked List

Task Statement: Define the Node structure with int data and Node* next. In main(), create only the head pointer and initialize it to nullptr. Print a message depending on whether the list is empty.

Program:

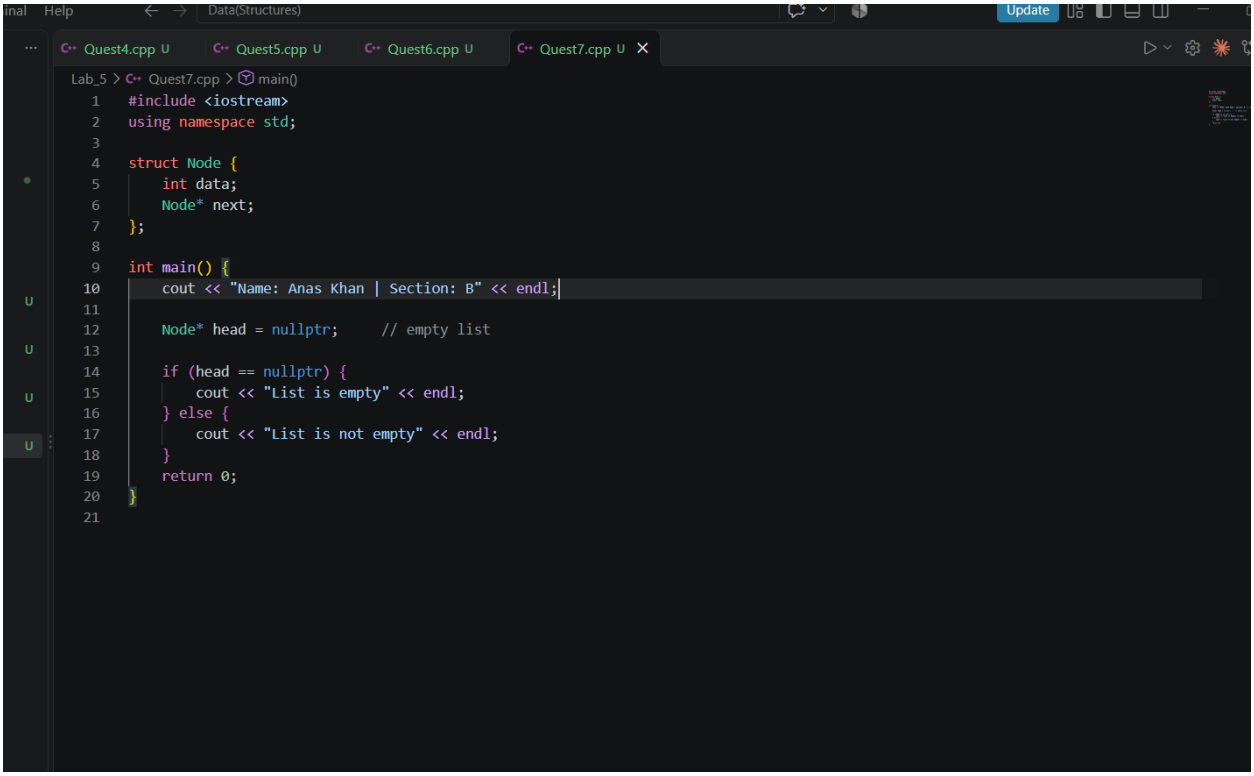
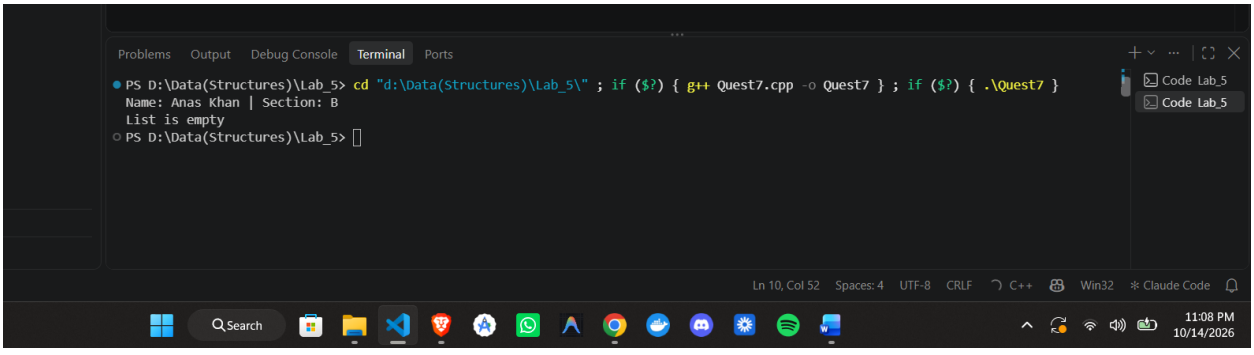


Diagram:

```
head ──────▶ nullptr
```

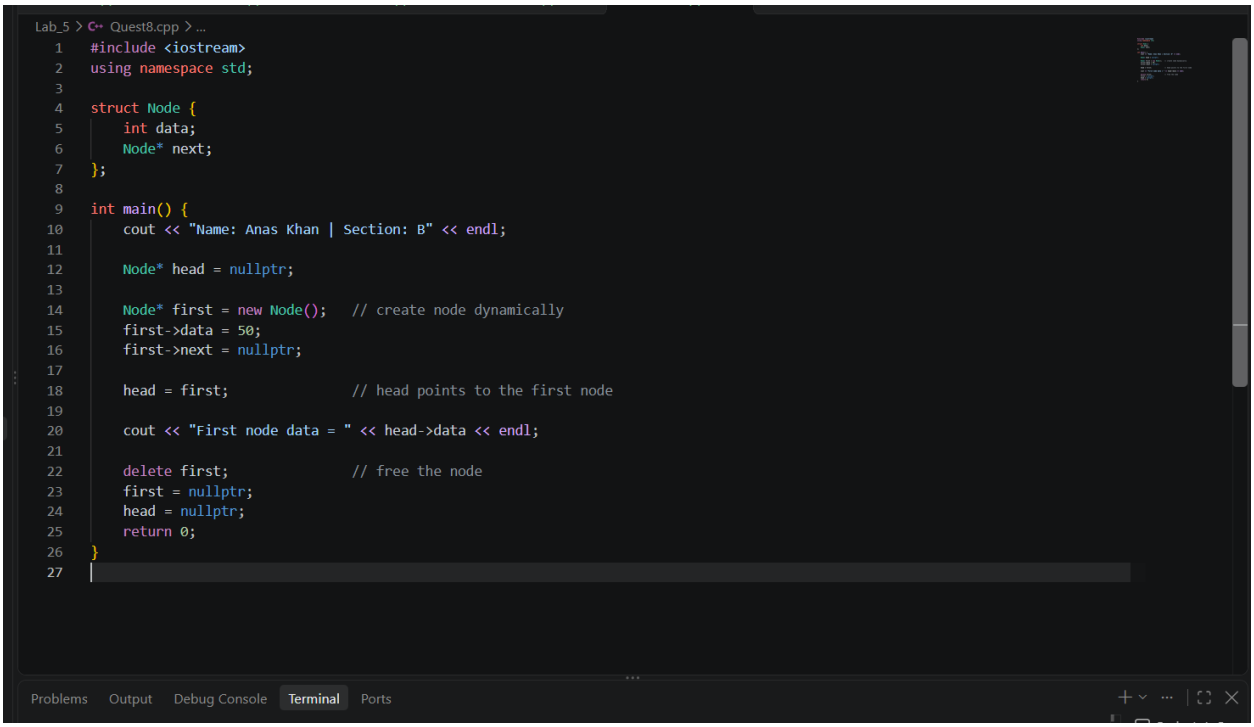
Output Screenshot:



TASK 5: Create a One-Node List

Task Statement: Create one node containing the value 50, set its next pointer to nullptr, make head point to it, and display the value using head->data. Free the node afterwards.

Program:



```cpp
#include <iostream>
using namespace std;

struct Node {
    int data;
    Node* next;
};

int main() {
    cout << "Name: Anas Khan | Section: B" << endl;

    Node* head = nullptr;

    Node* first = new Node();   // create node dynamically
    first->data = 50;
    first->next = nullptr;

    head = first;               // head points to the first node

    cout << "First node data = " << head->data << endl;

    delete first;               // free the node
    first = nullptr;
    head = nullptr;
    return 0;
}
```

Diagram:

```
head ──▶ [ 50 | nullptr ]
```

Output Screenshot:

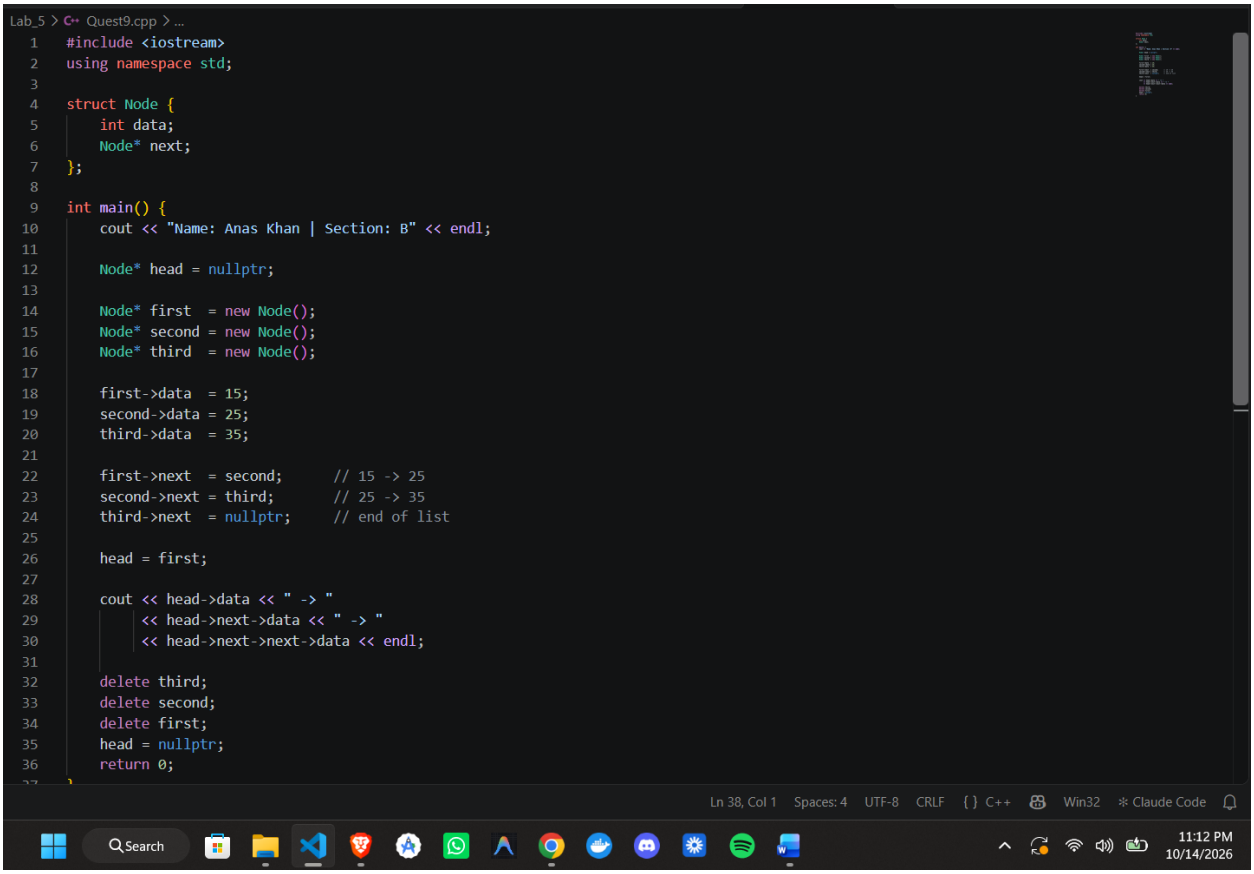


```
PS D:\Data(Structures)\Lab_5> cd "d:\Data(Structures)\Lab_5\" ; if ($?) { g++ Quest8.cpp -o Quest8 } ; if ($?) { .\Quest8 }
Name: Anas Khan | Section: B
First node data = 50
PS D:\Data(Structures)\Lab_5>
```

TASK 6: Build a Three-Node Linked List

Task Statement: Dynamically create three nodes containing 15, 25, and 35. Connect them in that order, make head point to the first node, and display all three values.

Program:



```
#include <iostream>
using namespace std;

struct Node {
    int data;
    Node* next;
};

int main() {
    cout << "Name: Anas Khan | Section: B" << endl;

    Node* head = nullptr;

    Node* first  = new Node();
    Node* second = new Node();
    Node* third  = new Node();

    first->data  = 15;
    second->data = 25;
    third->data  = 35;

    first->next  = second;      // 15 -> 25
    second->next = third;       // 25 -> 35
    third->next  = nullptr;     // end of list

    head = first;

    cout << head->data << " -> "
         << head->next->data << " -> "
         << head->next->next->data << endl;

    delete third;
    delete second;
    delete first;
    head = nullptr;
    return 0;
}
```

```
15 -> 25 -> 35
```

Three-Node Pointer Diagram:

```
  head ──▶ [ 15 | • ]──▶ [ 25 | • ]──▶ [ 35 | nullptr ]
           first          second         third
```

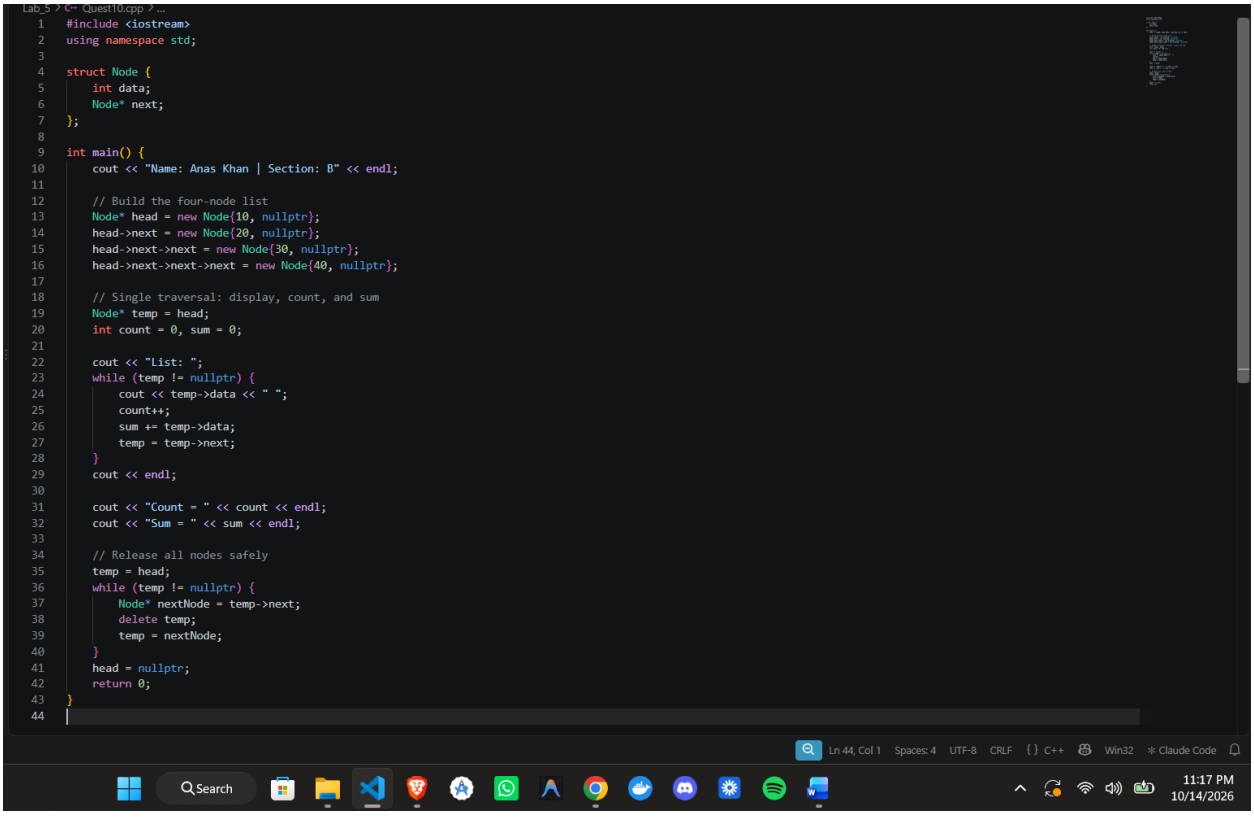
Output Screenshot:



TASK 7: Traverse, Count, and Sum

Task Statement: Create a four-node list containing 10, 20, 30, and 40. Traverse the list once to display all values, count the nodes, and calculate the sum.

Program:



```cpp
#include <iostream>
using namespace std;

struct Node {
    int data;
    Node* next;
};

int main() {
    cout << "Name: Anas Khan | Section: B" << endl;

    // Build the four-node list
    Node* head = new Node{10, nullptr};
    head->next = new Node{20, nullptr};
    head->next->next = new Node{30, nullptr};
    head->next->next->next = new Node{40, nullptr};

    // Single traversal: display, count, and sum
    Node* temp = head;
    int count = 0, sum = 0;

    cout << "List: ";
    while (temp != nullptr) {
        cout << temp->data << " ";
        count++;
        sum += temp->data;
        temp = temp->next;
    }
    cout << endl;

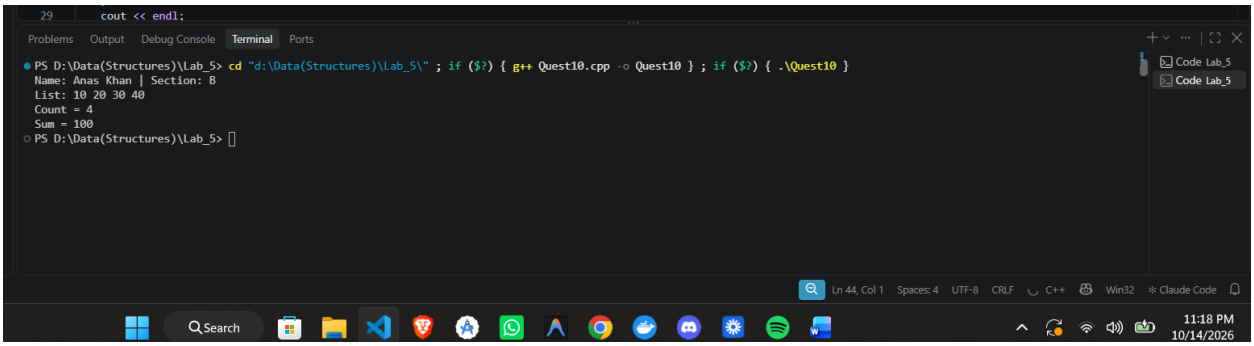
    cout << "Count = " << count << endl;
    cout << "Sum = " << sum << endl;

    // Release all nodes safely
    temp = head;
    while (temp != nullptr) {
        Node* nextNode = temp->next;
        delete temp;
        temp = nextNode;
    }
    head = nullptr;
    return 0;
}
```

Traversal Dry-Run Table:

Iteration	temp points to	temp->data	count	sum	temp moves to
Start	head (node 1)	–	0	0	–
1	node 1	10	1	10	node 2
2	node 2	20	2	30	node 3
3	node 3	30	3	60	node 4
4	node 4	40	4	100	nullptr
Stop	nullptr	–	4	100	loop ends

Output Screenshot:



End of Lab Task 5 — Submitted by Anas Khan, Section B, UET Peshawar